

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA
MACROAREA DI SCIENZE MATEMATICHE, FISICHE E NATURALI



CORSO DI LAUREA IN
INFORMATICA

TITOLO
Gli alberi Euler-Tour: analisi e implementazione

Relatore:
Chiar.mo Prof.
Luciano Gualà

Candidato:
Vittorio Porri
Matricola: 0323885

Anno Accademico 2024/2025

Indice

Indice	3
1 Introduzione	1
2 Il Problema della Connettività Dinamica su foreste	3
2.1 Operazioni	4
3 Struttura Dati	7
3.1 Alberi Binari di Ricerca	7
3.1.1 Operazioni	7
3.2 Red-Black Trees	11
4 Gli alberi Euler-Tour	17
4.1 Size	17
4.2 Rappresentazione del Tour	19
4.3 Operazioni	20
4.4 Codice	23
Bibliography	27

In questa tesi esploreremo gli Euler Tour Trees (ETT), una struttura dati dinamica introdotta da Henzinger e King [1], progettata specificamente per mantenere e gestire informazioni su foreste soggette a continue modifiche strutturali. L'obiettivo fondamentale di questa architettura è supportare in modo altamente efficiente una serie di operazioni topologiche essenziali:

- ▶ $\text{link}(u, v)$: permette di aggiungere un arco (u, v) , unendo due alberi disgiunti.
- ▶ $\text{cut}(u, v)$: consente di rimuovere l'arco (u, v) , dividendo l'albero di partenza in due alberi.
- ▶ $\text{isConnected}(u, v)$: verifica se i nodi u e v appartengono allo stesso albero della foresta.

Il presente lavoro di tesi si propone di sistematizzare ed estendere il framework teorico relativo alla struttura dati originale. L'obiettivo principale è stato quello di colmare le lacune informative e le omissioni che, per inevitabile mancanza di spazio, si riscontrano nell'articolo scientifico originale. Tali elementi sono stati qui rianalizzati e descritti in maniera esaustiva e rigorosa, formalizzando tutti i dettagli algoritmici e strutturali necessari alla piena comprensione del funzionamento degli ETT. Un elemento di forte originalità e rilevanza teorica di questa tesi risiede nella scelta dell'architettura sottostante. A differenza delle implementazioni convenzionali presenti in letteratura, storicamente basate sugli Splay Tree, in questo lavoro si è scelto di adottare i Red-Black Tree. Tale approccio garantisce il mantenimento di un costo computazionale logaritmico esplicito nel caso peggiore ($O(\log n)$), superando il vincolo del costo ammortizzato tipico delle strutture auto-bilancianti classiche. Tuttavia, per adattare questa tipologia di alberi alle esigenze specifiche degli Euler Tour Trees, si è reso necessario estendere la struttura dati standard introducendo due operazioni non convenzionali per i Red-Black Tree: la *join* e la *split* [2]. La trattazione formale e la minuziosa scomposizione di ogni singola operazione d'alto livello in primitive di manipolazione dei Red-Black Tree non si sono limitate a un puro esercizio teorico. Al contrario, la rigorosa esaustività della descrizione ha costituito la base imprescindibile che ha permesso l'implementazione pratica dell'intera struttura dati in linguaggio C++.

Il Problema della Connettività Dinamica su foreste

2

Il problema della connettività dinamica consiste nel mantenere aggiornate le informazioni sulla struttura di un grafo soggetto a modifiche topologiche repentine. Nel caso specifico delle foreste, l'obiettivo è la gestione di una collezione di alberi disgiunti tramite le operazioni di:

- **link(u,v)**: aggiunge un arco (u, v) che unisce due alberi
- **cut(u,v)**: rimuove l'arco (u, v) dividendo l'albero originale in due

Definition 2.0.1 Una foresta è un grafo aciclico non orientato

Per gestire queste operazioni in modo efficiente, Henzinger e King [1] introducono gli Euler Tour Trees (ETT). Questa struttura dati a differenza di altre strutture che operano direttamente sui nodi, si basa sulla linearizzazione dell'albero tramite cicli di Eulero. Questa rappresentazione permette di ridurre complessi problemi di connettività a semplici manipolazioni di sequenze.

Definition 2.0.2 Un ciclo euleriano è un percorso in un grafo che inizia e finisce nello stesso nodo passando esattamente una volta per ogni arco.

Note

Poiché gli alberi non hanno veri cicli euleriani, consideriamo ogni arco $e = (u, v)$ come una coppia di archi diretti (u, v) e (v, u) .

L'obiettivo della struttura dati è rappresentare un albero radicato tramite il suo Tour Euleriano. Questo viene generato eseguendo una visita in profondità (DFS) e tracciando la sequenza temporale di tutti gli archi attraversati

Algorithm 1: Creazione Tour Euleriano

Input: Nodo corrente u , Lista L

Output: Cammino euleriano in L

```
1 foreach figlio  $v$  di  $u$  do
2    $L.append((u, v))$ 
3   makeTour( $v, L$ )
4    $L.append((v, u))$ 
5 end
```

Il ciclo euleriano è strettamente connesso con la struttura dell'albero; ad esempio si osserva che la sequenza evidenziata (A,B) , (B,D) , (D,B) , (B,E) , (E,B) , (B,A) , (A,C) , (C,A) rappresenta il sottoalbero radicato in B

Claim

Sia S la sequenza di archi che descrive il tour euleriano per ogni nodo v il sottoalbero radicato in v è rappresentato dall'intervallo contiguo che va dal primo arco che ha $(v, _)$ fino all'ultimo arco che contiene $(_, v)$

2.1 Operazioni 4

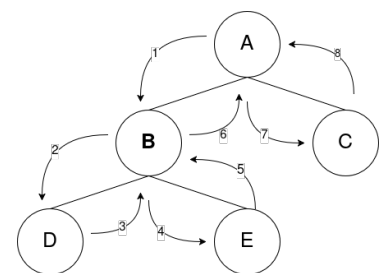


Figura 2.1: Il Ciclo euleriano che si forma è (A,B) , (B,D) , (D,B) , (B,E) , (E,B) , (B,A) , (A,C) , (C,A)

2.1 Operazioni

In questa sezione verranno descritte, ad un alto livello di astrazione, le modalità di implementazione delle operazioni fondamentali sugli Euler Tour Tree (ET-tree), modellandole direttamente come manipolazioni dei cicli euleriani corrispondenti. Questo approccio permetterà di comprendere la logica strutturale delle operazioni indipendentemente dai dettagli tecnologici. Successivamente, nel capitolo seguente, verrà mostrato come tali operazioni possano essere tradotte in algoritmi efficienti, formalizzando la rappresentazione dei cicli euleriani mediante l'utilizzo di opportuni alberi di ricerca bilanciati.

isConnected

L'operazione **isConnected(u,v)** è la più semplice poichè è sufficiente verificare che i due nodi u e v si trovino nello stesso ciclo euleriano, per farlo basta verificare che il primo elemento della sequenza sia il medesimo per i due nodi.

Rerooting

Prima di introdurre le altre operazioni introduciamo un'operazione ausiliaria **rerooting(T,v)**, tale operazione ha il compito di ri-radicare l'albero rispetto ad un nodo interno, per farlo seguiamo i seguenti passaggi:

1. si cerca una occorrenza che ha (v, c) (dove c è un figlio casuale di v) nel ciclo euleriano, e si separa la sequenza in A che conterrà tutto ciò che c'era prima di tale arco e B che conterrà l'arco (v, c) e tutto ciò che c'è dopo
2. riconcateno $B + A$

Algorithm 2: Rerooting

Input: ETT L , nodo v

Output: Il nuovo ETT con radice v

```

1 if  $L[0].source == v$  then
2   return  $L$ 
3 end
4  $A \leftarrow \emptyset; B \leftarrow \emptyset; isFound \leftarrow False; c \leftarrow NULL$ 
5 for  $i \leftarrow 0$  to  $len(L) - 1$  do
6   // Cerchiamo il primo arco che parte da  $v$ 
7   if  $L[i].source == v$  and  $\neg isFound$  then
8      $isFound \leftarrow True$ 
9      $c \leftarrow L[i].target$ 
10  end
11  if  $\neg isFound$  then
12     $A \leftarrow A + L[i]$ 
13  end
14  else
15     $B \leftarrow B + L[i]$ 
16  end
17 end
18 return  $B + A$ 

```

Link

L'operazione **link**(u, v) permette di unire due alberi distinti aggiungendo un arco tra il nodo u (appartenente al primo albero) e il nodo v (appartenente al secondo), per farlo seguiamo i seguenti passaggi:

1. $A = \text{rerooting}(u)$
2. $B = \text{rerooting}(v)$
3. concateno $A + (u, v) + B + (v, u)$

Algorithm 3: Link

Input: ETT L , ETT C , nodo u , nodo v

Output: l'ETT del nuovo albero

- 1 $A \leftarrow \text{rerooting}(L, u)$
 - 2 $B \leftarrow \text{rerooting}(C, v)$
 - 3 **return** $A + (u, v) + B + (v, u)$
-

Cut

L'operazione **cut**(u, v) permette di separare un albero in due alberi distinti. Eliminando l'arco (u, v) creiamo due nuovi alberi radicati rispettivamente in u e v , per farlo seguiamo i seguenti passaggi:

1. $\text{rerooting}(v)$
2. si cerca la sequenza (u, v) e (v, u) e si separa in A che conterrà tutto ciò che c'era prima di (u, v) , B che non conterrà (u, v) e (v, u) ma conterrà unicamente ciò che c'è tra i due e C che contiene ciò che c'è dopo (v, u)
3. si eliminano il primo e l'ultimo elemento in B
4. si restituisce $A + C$ e B

Remark 2.1.1 Tutte queste implementazioni delle operazioni però hanno costo proporzionale alla dimensione del ciclo euleriano, dunque il loro costo sarà $O(m)$

Introduciamo una struttura dati che ci permetterà di eseguire queste operazioni in maniera molto più efficiente.

Algorithm 4: Cut

Input: ETT L , nodo u , nodo v **Output:** Due ETT: $L_{\text{rimanente}}$ e L_{staccato}

```

1  $L \leftarrow \text{rerooting}(L, v)$ 
2  $A \leftarrow \emptyset; B \leftarrow \emptyset; C \leftarrow \emptyset$ 
  // Trovo il primo arco in cui compare  $(u, -)$ 
3 for  $i \leftarrow 0$  to  $\text{len}(L) - 1$  do
4   if  $L[i].\text{source} == u$  then
5     break
6   end
7    $A \leftarrow A + L[i]$ 
8 end
  // Trova l'ultima arco in cui compare  $(-, u)$ 
9 for  $j \leftarrow \text{len}(L) - 1$  to  $0$  do
10  if  $L[j].\text{target} == u$  then
11    break
12  end
13   $C \leftarrow L[j] + C$ 
14 end
  // Costruisce la sequenza B tra le due occorrenze
15 for  $k \leftarrow i$  to  $j$  do
16    $B \leftarrow B + L[k]$ 
17 end
18 elimino il primo e l'ultimo elemento di  $B$ 
19 return  $(A + C), B$ 

```

3.1 Alberi Binari di Ricerca

3.1 Alberi Binari di Ricerca	7
3.1.1 Operazioni	7
3.2 Red-Black Trees	11

Un Albero Binario di Ricerca (BST) è una struttura dati ad albero che soddisfa le seguenti proprietà per ogni nodo v :

- Ogni nodo contiene un elemento a cui è associata una chiave presa da un dominio totalmente ordinato.
- Tutte le chiavi nel sottoalbero **sinistro** di v sono minori o uguali ($\leq$) alla chiave in v .
- Tutte le chiavi nel sottoalbero **destro** di v sono maggiori o uguali ($\geq$) alla chiave in v .

3.1.1 Operazioni

Search

L'algoritmo di ricerca ricalca il principio della ricerca binaria. Partendo dalla radice, si confronta la chiave cercata k con la chiave del nodo corrente v :

- Se $k = \text{key}(v)$, la ricerca termina con successo e ritorna.
- Se $k < \text{key}(v)$, la ricerca prosegue ricorsivamente nel sottoalbero **sinistro**.
- Se $k > \text{key}(v)$, la ricerca prosegue ricorsivamente nel sottoalbero **destro**.

Se si raggiunge un nodo nullo (NULL), la chiave non è presente nell'albero.

Algorithm 5: Ricerca in un BST

Input: Nodo radice u , chiave cercata k

Output: Il nodo con chiave k o NULL

```
1 while  $u \neq \text{NULL}$  do
2   if  $k == u.\text{key}$  then
3     return  $u$ 
4   end
5   if  $k < u.\text{key}$  then
6      $u \leftarrow u.\text{left}$ 
7   end
8   else
9      $u \leftarrow u.\text{right}$ 
10  end
11 end
12 return NULL
```

Insert

L'obiettivo dell'inserimento è posizionare un nuovo nodo u come **foglia**, preservando le proprietà dell'albero. Il procedimento prevede i seguenti passi:

1. Si crea il nodo u con i relativi attributi ($elem, key$).
2. Si simula una procedura di $search(key(u))$ per individuare il futuro nodo padre.
3. Una volta individuato il padre, si aggancia u come figlio destro o sinistro in modo da rispettare le proprietà dei BST.

Algorithm 6: Inserimento in un BST

Input: Nodo radice r , nuovo nodo u (già inizializzato)

```

1  $y \leftarrow \text{NULL}$                                      // Padre di  $u$ 
2 if  $r == \text{NULL}$  then
3   |  $r \leftarrow u$                                      // Albero vuoto
4 end
5 else
6   |  $x \leftarrow r$ 
7   while  $x \neq \text{NULL}$  do
8     |  $y \leftarrow x$ 
9     | if  $u.key < x.key$  then
10    | |  $x \leftarrow x.left$ 
11    | end
12    | else
13    | |  $x \leftarrow x.right$ 
14    | end
15  end
16   $u.parent \leftarrow y$ 
17  if  $u.key < y.key$  then
18    |  $y.left \leftarrow u$ 
19  end
20  else
21    |  $y.right \leftarrow u$ 
22  end
23 end

```

Delete

L'operazione di cancellazione di un nodo v è la più complessa da implementare, quindi prima di definirla è necessario introdurre alcune operazioni ausiliarie fondamentali.

- **Ricerca del minimo (findMin):** Partendo dal nodo radice, si segue ricorsivamente il puntatore **sinistro**. Il minimo è rappresentato dal primo nodo incontrato il cui puntatore sinistro è **NULL**.
- **Ricerca del successore (findSuccessor):** Il successore di un nodo v è il nodo con la minima chiave tra tutti i nodi aventi chiave maggiore di $key(v)$. Si distinguono due casi:
 1. **Se v ha un sottoalbero destro:** il successore è il valore minimo di tale sottoalbero ($findMin(v.right)$).
 2. **Se v non ha un figlio destro:** il successore si trova risalendo l'albero fino a individuare il primo antenato di v tale che v appartenga al suo sottoalbero **sinistro**.

Algorithm 7: Ricerca del minimo in un BST

Input: Nodo radice r **Output:** Il nodo con chiave minima o NULL

```

1 if  $r == \text{NULL}$  then
2   | return NULL
3 end
4  $x \leftarrow r$ 
5 while  $x.\text{left} \neq \text{NULL}$  do
6   |  $x \leftarrow x.\text{left}$ 
7 end
8 return  $x$ 

```

Algorithm 8: Ricerca del successore di un nodo in un BST

Input: Nodo v di cui cercare il successore**Output:** Il nodo successore o NULL

```

1 if  $v.\text{right} \neq \text{NULL}$  then
2   | return findMin( $v.\text{right}$ )
3 end
4  $u \leftarrow v$ 
5 while  $u.\text{parent} \neq \text{NULL} \wedge u.\text{parent}.\text{right} == u$  do
6   |  $u \leftarrow u.\text{parent}$ 
7 end
8 return  $u.\text{parent}$ 

```

Le operazioni di cancellazione richiedono la gestione di tre scenari differenti:

- **Caso 1: v è una foglia.** Si rimuove il nodo aggiornando il corrispondente puntatore del padre a NULL.
- **Caso 2: v ha un solo figlio.** Si effettua un'operazione di "scavalcamento": il padre di v viene collegato direttamente all'unico figlio di v , rimuovendo v dalla struttura.
- **Caso 3: v ha due figli.** Non è possibile rimuovere v direttamente. Si procede come segue:

1. Si individua il **successore** di v (che si troverà nel sottoalbero destro).
2. Si copia il contenuto (chiave ed elementi) del successore nel nodo v .
3. Si rimuove il successore dall'albero. Avendo al massimo un figlio (il destro), la sua rimozione ricadrà nel Caso 1 o nel Caso 2.

Algorithm 9: Cancellazione di un nodo in un BST**Input:** Nodo radice r , nodo v da rimuovere

```

1  $p \leftarrow v.parent$ 
2 if  $v.left == NULL \wedge v.right == NULL$  then
3   if  $p == NULL$  then
4      $r \leftarrow NULL$ 
5   end
6   else if  $p.left == v$  then
7      $p.left \leftarrow NULL$ 
8   end
9   else
10     $p.right \leftarrow NULL$ 
11  end
12 end
13 else if  $(v.left == NULL) \text{ xor } (v.right == NULL)$  then
14   if  $v.left \neq NULL$  then
15      $child \leftarrow v.left$ 
16   end
17   else
18      $child \leftarrow v.right$ 
19   end
20   if  $p == NULL$  then
21      $r \leftarrow child$ 
22   end
23   else if  $p.left == v$  then
24      $p.left \leftarrow child$ 
25   end
26   else
27      $p.right \leftarrow child$ 
28   end
29   if  $child \neq NULL$  then
30      $child.parent \leftarrow p$ 
31   end
32 end
33 else
34    $s \leftarrow \text{findSuccessor}(v)$ 
35    $v.key \leftarrow s.key$ 
36    $v.elem \leftarrow s.elem$ 
37   Delete( $r, s$ )
38 end

```

Note

Tutte le operazioni dipendono dall'altezza dell'albero per cui il loro costo è $O(h)$.

3.2 Red-Black Trees

Lo standard per garantire l'efficienza di queste operazioni è imporre una *condizione di bilanciamento*. Tale vincolo assicura che l'altezza dell'albero rimanga logaritmica rispetto al numero di nodi ($h = O(\log n)$), evitando la degenerazione della struttura in una lista concatenata. Per farlo, ad ogni nodo viene associata una funzione chiamata **rango** che assegna ad ogni nodo x un valore intero non negativo $rank(x)$ che soddisfa le seguenti proprietà fondamentali:

1. **Proprietà del Padre:** Se x è un nodo dotato di genitore $p(x)$, allora:

$$rank(x) \leq rank(p(x)) \leq rank(x) + 1$$

2. **Proprietà del Nonno:** Se x è un nodo dotato di un nonno $p^2(x)$, allora:

$$rank(x) < rank(p^2(x))$$

3. **Proprietà dei Nodi Esterni:** Se x è un nodo esterno (NULL), allora $rank(x) = 0$. Inoltre, se x possiede un genitore, allora $rank(p(x)) = 1$.

Come visto in [2] è possibile derivare una definizione alternativa basata sulla colorazione dei nodi. Si definisce un nodo x come **Nero** (Black) se $rank(p(x)) = rank(x) + 1$ (o se x è la radice) e **Rosso** (Red) se $rank(p(x)) = rank(x)$.

Sotto questa formalizzazione, le condizioni si traducono nelle proprietà canoniche dei **Red-Black Trees**:

1. Ogni nodo è marcato con il colore rosso o nero.
2. Un nodo rosso deve avere genitore e figli di colore nero.
3. Ogni nodo esterno è nero e tutti i cammini che congiungono un nodo x ai nodi esterni discendenti contengono il medesimo numero di nodi neri.
4. La radice dell'albero è nera.

Note

$rank(v) = k$ è il numero di nodi neri che si incontrano dal nodo v , escluso, verso qualsiasi sua foglia.

Lemma 3.2.1 *Un albero binario bilanciato con n nodi interni ha altezza $h \leq 2 \log_2(n + 1)$.*

Dimostrazione. Dimostriamo per induzione sul rango k che un nodo u con $rank(u) = k$ possiede un numero di discendenti interni $n(u) \geq 2^k - 1$.

- **Caso base** ($k = 0$): per la Proprietà dei Nodi Esterni, un nodo con rango 0 è esterno. Esso non possiede discendenti interni: $2^0 - 1 = 0$.
- **Passo induttivo** ($k > 0$): Supponiamo la proprietà vera per ranghi inferiori a k . Sia u un nodo con $rank(u) = k$ e siano v e w i suoi figli. Per la *Proprietà del Padre*, il rango di un figlio può essere k o $k - 1$. Analizziamo un generico figlio v :

1. Se $rank(v) = k - 1$, per ipotesi induttiva v ha almeno $2^{k-1} - 1$ discendenti.

2. Se $\text{rank}(v) = k$, allora v è rosso. Per la *Proprietà del Nonno*, i figli di v devono avere rango $k - 1$. Applicando l'ipotesi induttiva ai due figli di v , il numero di discendenti di v è almeno $(2^{k-1} - 1) + (2^{k-1} - 1) + 1 = 2^k - 1$.

In entrambi i casi, ogni figlio di u garantisce almeno $2^{k-1} - 1$ discendenti.

Sommando i contributi dei due figli di u otteniamo:

$$n(u) \geq (2^{k-1} - 1) + (2^{k-1} - 1) + 1 = 2^k - 1$$

La proprietà è dimostrata per ogni k .

Sia T un albero con n nodi interni e sia k il rango della sua radice. Sappiamo che:

$$n \geq 2^k - 1 \implies n + 1 \geq 2^k \implies \log_2(n + 1) \geq k$$

Dalla *Proprietà del Nonno*, lungo ciascun cammino non possono esserci due nodi rossi consecutivi. Questo implica che l'altezza fisica h è al massimo il doppio del rango della radice ($h \leq 2k$).

Combinando le disuguaglianze:

$$h \leq 2k \leq 2 \log_2(n + 1)$$

□

Le operazioni di inserimento (Insert) e cancellazione (Delete) ereditano la logica dei BST, tuttavia potrebbero invalidare il bilanciamento cromatica. Introduciamo quindi le procedure di ripristino.

Rotazioni e Ribilanciamento

Le rotazioni sono operazioni locali che modificano la struttura preservando l'ordine simmetrico dei nodi. Un nodo v è sbilanciato se uno dei sottoalberi è troppo profondo rispetto agli altri.

Sia v il nodo sbilanciato; distinguiamo quattro casi in base alla posizione del sottoalbero T critico:

Caso Sinistro-Sinistro (SS): T è figlio sinistro del figlio sinistro di v . Si risolve con una *rotazione semplice a destra* su v .

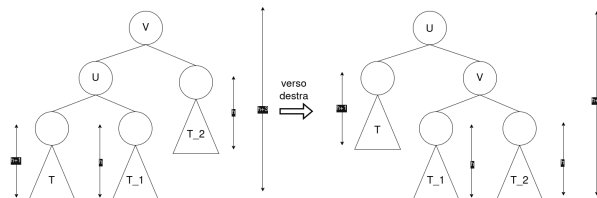


Figura 3.1: Rotazione semplice destra (Caso SS).

Caso Destro-Destro (DD): Speculare al caso SS. Si risolve con una *rotazione semplice a sinistra* su v .

Caso Sinistro-Destro (SD): T è figlio destro del figlio sinistro di v . Si risolve con una *rotazione doppia* (sinistra sul figlio, poi destra su v).

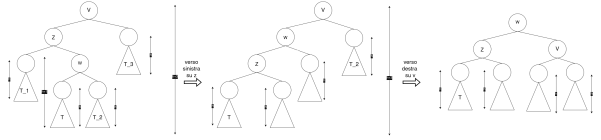


Figura 3.2: Rotazione doppia sinistra-destra (Caso SD).

Caso Destro-Sinistro (DS): Speculare al caso SD. Si risolve con una rotazione doppia destra-sinistra.

Lemma 3.2.2 Sia v un nodo sbilanciato a seguito di un inserimento o cancellazione. Esiste sempre una rotazione semplice o doppia in grado di ripristinare il bilanciamento del sottoalbero radicato in v .

Dimostrazione. Consideriamo il caso Sinistro-Sinistro (Fig. 3.1). Prima della rotazione, l'altezza del sottoalbero radicato in v è $h + 3$. Dopo la rotazione semplice destra, il nodo u diventa la nuova radice locale e la nuova altezza scende a $h + 2$. Ragionamento analogo si applica alla rotazione doppia (Fig. 3.2), dove la sequenza trasforma un cammino critico di altezza $h + 3$ in una struttura bilanciata di altezza $h + 2$. L'altezza viene ridotta esattamente del valore necessario a compensare lo sbilanciamento preservando l'ordinamento. $\square$

Insert

L'inserimento in un *Red-Black Tree* (RBT) integra la logica strutturale dei BST con un protocollo di ripristino delle proprietà cromatiche. Si inserisce il nuovo nodo v colorato di **rosso** seguendo la procedura standard dei BST.

Analisi della politica di colorazione La scelta del colore rosso limita le potenziali violazioni alle proprietà strutturali:

- **Invarianza dell'Altezza Nera (Proprietà 3):** L'assegnazione del rosso garantisce il mantenimento della *Proprietà 3*. Poiché v è rosso, il conteggio dei nodi neri (*black-height*) per ogni cammino che lo attraversa rimane inalterato.
- **Vincoli di Adiacenza (Proprietà 2):** Le uniche potenziali violazioni riguardano la presenza di due nodi rossi consecutivi. Tale conflitto si manifesta esclusivamente se il padre $p(v)$ è rosso; in questo scenario si invoca la procedura `fixInsert`.

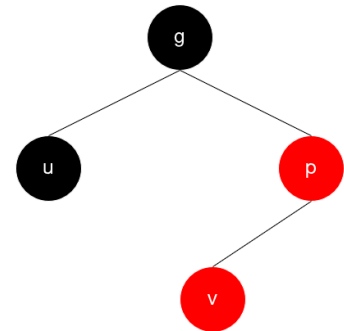


Figura 3.3: Caso del triangolo destro

fixInsert

La routine `fixInsert(v)` interviene quando il padre $p[v]$ è rosso, violando la *Proprietà 2*. Il comportamento dipende dal colore e dalla posizione dello zio u (fratello di $p[v]$). Distinguiamo i seguenti casi fondamentali:

1. **Caso della Radice:** Se $p[v]$ è NULL, v è la radice. Si ricolore v di nero.
2. **Ricolorazione (Zio Rosso):** Se lo zio u è rosso, il nonno g deve essere nero. Si ricolore $p[v]$ e u di nero, e g di rosso. La procedura viene poi chiamata ricorsivamente su g .

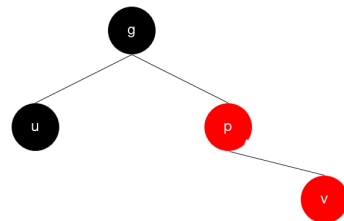


Figura 3.4: Caso della linea destra

3. **Caso del Triangolo:** Se lo zio u è nero (o NULL) e v è un figlio interno rispetto al nonno (Fig. 3.3), si applica una **rotazione semplice** sul padre $p[v]$ per ricondursi al caso della linea.
4. **Caso della Linea:** Se lo zio u è nero (o NULL) e v è un figlio esterno (Fig. 3.4), si applica una **rotazione semplice** sul nonno g , ricolorando $p[v]$ di nero e g di rosso. Questa operazione risolve la violazione senza alterare la *black-height*.

Remark 3.2.1 Mentre il Caso 2 può propagare la violazione fino alla radice con costo $O(\log n)$, i Casi 3 e 4 richiedono al massimo due rotazioni totali per terminare.

Delete

La rimozione segue inizialmente la logica dei BST. Le implicazioni cromatiche variano in base al colore del nodo rimosso o del sostituto:

- Se il nodo rimosso è **rosso**, le proprietà rimangono invariate.
- Se il nodo rimosso è **nero**, si viola la *Proprietà 3*, riducendo di un'unità il numero di nodi neri sui cammini interessati.

Per ovviare a ciò, si invoca la procedura `fixDelete` sul nodo che sostituisce fisicamente quello eliminato.

fixDelete

La procedura `fixDelete(x)` ripristina la *black-height* risalendo l'albero o assorbendo la violazione tramite rotazioni. Il comportamento è determinato dal colore del fratello f e dei suoi figli cl (sinistro) e cr (destro).

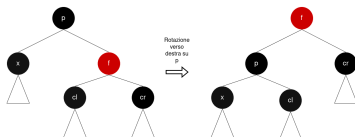


Figura 3.5: Caso del fratello Rosso

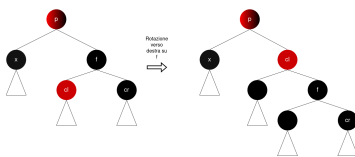


Figura 3.6: Caso del nipote vicino rosso

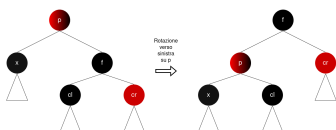


Figura 3.7: Caso del nipote lontano rosso

1. **Caso 1: Fratello f rosso.** Si esegue una rotazione su $p[x]$ (Fig. 3.5) e si scambiano i colori tra $p[x]$ e f . Ci si riconduce così ai casi 2, 3 o 4.
2. **Caso 2: Fratello f e nipoti neri.** Vengono ricolorati x e f di rosso, verificando poi $p[x]$: se $p[x]$ era rosso diventa nero e si termina, altrimenti si risale ricorsivamente su $p[x]$.
3. **Caso 3: Nipote "vicino" rosso.** Si verifica quando il figlio di f più prossimo a x è rosso (Fig. 3.6). Si esegue una rotazione su f in direzione opposta a x e si scambiano i colori tra f e il nipote, riconducendosi al Caso 4.
4. **Caso 4: Nipote "lontano" rosso.** Si verifica quando il figlio di f più distante da x è rosso (Fig. 3.7). Si ruota su $p[x]$ verso x : f prende il colore originale di $p[x]$, mentre $p[x]$ e il nipote diventano neri.

Note

Nei casi 3 e 4, il colore di p può essere sia rosso che nero.

Remark 3.2.2 Il Caso 2 può propagarsi fino alla radice in $O(\log n)$, mentre i Casi 3 e 4 garantiscono la terminazione in tempo costante $O(1)$.

Join

L'operazione di **Join** riceve in input due alberi Red-Black S_1 e S_2 e un nodo con chiave i , sotto l'ipotesi che:

$$\forall x \in S_1, \forall y \in S_2 : \text{key}(x) < i < \text{key}(y)$$

L'algoritmo si divide in due casi speculari:

- **Caso** $\text{rank}(S_1) \geq \text{rank}(S_2)$:
 1. **Ricerca**: Si discende lungo il cammino destro di S_1 fino a un nodo x con $\text{rank}(x) = \text{rank}(S_2)$.
 2. **Innesto**: Sia p il padre di x . Il nodo i diventa il nuovo figlio destro di p , con figli x (sinistro) e la radice di S_2 (destro).
 3. **Colorazione**: i viene colorato di **rosso**. Le radici x e S_2 vengono ricolorate di **nero**.
 4. **Ribilanciamento**: Viene invocata la procedura $\text{fixInsert}(i)$.
- **Caso** $\text{rank}(S_1) < \text{rank}(S_2)$: Procedimento simmetrico discendendo lungo il cammino sinistro di S_2 fino a trovare $\text{rank}(x) = \text{rank}(S_1)$.

Definition 3.2.1 Il costo del $\text{join}(S_1, i, S_2)$ è $O(\log n)$.

Dimostrazione. La ricerca richiede tempo proporzionale a $|\text{rank}(S_1) - \text{rank}(S_2)|$. L'innesto richiede $O(1)$ e la routine fixInsert impiega un tempo limitato dalla differenza di rango. Il costo totale è quindi $O(|\text{rank}(S_1) - \text{rank}(S_2)| + 1) = O(\log n)$. $\square$

Note

Il rango dell'albero risultante da $\text{join}(S_1, i, S_2)$ sarà:

- $\text{rank}(S_1) + 1$ se $\text{rank}(S_1) = \text{rank}(S_2)$.
- $\max(\text{rank}(S_1), \text{rank}(S_2))$ se $\text{rank}(S_1) \neq \text{rank}(S_2)$.

Split

L'operazione di $\text{split}(T, v)$ divide l'albero T in due alberi T_L e T_R rispetto alla chiave del nodo v . La procedura avviene in due fasi:

1. Si individua il nodo v e si inizializzano $T_L = v.\text{left}$ e $T_R = v.\text{right}$.
2. Si risale l'albero verso la radice. Per ogni nodo padre p nel cammino:
 - Se v discende dal figlio sinistro di p : $T_R = \text{join}(T_R, p, p.\text{right})$.
 - Se v discende dal figlio destro di p : $T_L = \text{join}(p.\text{left}, p, T_L)$.

Definition 3.2.2 Il costo dello $\text{split}(T, v)$ è $O(\log n)$.

Dimostrazione. Ci concentriamo sull'accumulatore T_L (il ragionamento per T_R è simmetrico). Ad ogni passo i , se il cammino devia a destra eseguiamo $\text{Join}(S_1, n, T_{L,i})$, fondendo l'accumulato corrente con il sottoalbero sinistro S_i mediante il perno n .

Claim

Ad ogni passo i della risalita si ha: $\text{rank}(S_i) \geq \text{rank}(T_{L,i})$.

Dimostrazione. Per le proprietà dei Red-Black Trees, la differenza di rango tra un nodo e i suoi nipoti è al massimo 1. Analizziamo un nodo n :

- Se n è nero, i suoi figli hanno lo stesso rango e $\text{rank}(n.\text{parent}) = \text{rank}(n) + 1$. Il fratello S_i avrà $\text{rank}(S_i) = \text{rank}(n.\text{parent})$ (se rosso) oppure $\text{rank}(S_i) = \text{rank}(n)$ (se nero).
- Se n è rosso, i suoi figli hanno rango inferiore di 1, mentre $p(n)$ è nero con $\text{rank}(n.\text{parent}) = \text{rank}(n)$. Il fratello S_i avrà rango pari a $\text{rank}(n.\text{parent})$ (se rosso) o pari al rango dei figli di n (se nero).

In ogni scenario $\text{rank}(S_i)$ è maggiore o uguale al rango dei nipoti. Poiché la funzione `join` preserva il rango, per induzione concludiamo che $\text{rank}(S_i) \geq \text{rank}(T_{L,i})$. $\square$

Dal claim ricaviamo che $\text{rank}(T_{L,i+1}) \approx \text{rank}(S_i)$. Il costo di ogni singola operazione `joini` è:

$$\text{costo}(\text{join}_i) = O(\text{rank}(S_i) - \text{rank}(T_{L,i}) + 1) = O(\text{rank}(T_{L,i+1}) - \text{rank}(T_{L,i}) + 1)$$

Il costo totale dello `split` per T_L è dato dalla sommatoria:

$$\sum_{i=1}^j O(\text{rank}(T_{L,i+1}) - \text{rank}(T_{L,i}) + 1)$$

Espandendo la sommatoria telescopica:

$$\begin{aligned} \text{Costo} = O & \left((\text{rank}(T_{L,1}) - \text{rank}(T_{L,0})) + \dots \right. \\ & \left. + (\text{rank}(T_{L,j}) - \text{rank}(T_{L,j-1})) + \sum_{i=1}^j 1 \right) \end{aligned}$$

I termini intermedi si elidono, lasciando:

$$O(\text{rank}(T_{L,j}) - \text{rank}(T_{L,0}) + j)$$

Poiché $\text{rank}(T_{L,0}) \geq 0$ e il numero di passi j è limitato dall'altezza dell'albero ($j = O(\log n)$), la complessità finale è:

$$O(\log n - 0 + \log n) = O(\log n)$$

$\square$

4.1 Size

4.1 Size	17
4.2 Rappresentazione del Tour	19
4.3 Operazioni	20
4.4 Codice	23

Negli Euler Tour Trees, la struttura deve gestire sequenze dinamiche dove la posizione dei nodi cambia continuamente a causa di cut e link. Per trasformare i RB-Trees una struttura dati dinamica abbiamo bisogno di eliminare il concetto di chiavi statiche introducendone uno che gestisce le chiavi in maniera implicita.

L'idea fondamentale è che la posizione di un nodo nella sequenza non è determinata da un valore fisso, ma dalla sua posizione relativa all'interno dell'ETT. Per gestire efficientemente questa informazione, ogni nodo v memorizza un attributo aggiuntivo denominato **size** (tecnica formalizzata da Cormen in [3])

Definition 4.1.1 $size(v) = 1 + size(v.left) + size(v.right)$

Perche la size è fondamentale?

L'introduzione della size nei Red-Black Tree permette di:

- Calcolare la posizione di un nodo nella sequenza risalendo il RB-Trees verso la radice.

Algorithm 10: Posizione del nodo nella sequenza

Input: Nodo n

Output: Posizione i del nodo nell'ETT

```

1  $i \leftarrow size(n.left) + 1$ 
2 while  $n.parent \neq NULL$  do
3    $p \leftarrow n.parent$ 
4   if  $n == p.right$  then
5      $i \leftarrow i + size(p.left) + 1$ 
6   end
7    $n \leftarrow p$ 
8 end
9 return  $i$ 
```

- Individuare l'elemento in i -esima posizione discendendo nel RB-Trees basandomi sulla size.

Algorithm 11: Nodo in posizione i -esima**Input:** Radice dell'albero r , posizione desiderata i **Output:** Nodo n in posizione i nella sequenza

```

1  $n \leftarrow r$ 
2 while  $n \neq \text{NULL}$  do
3   if  $i == \text{size}(n.\text{left}) + 1$  then
4     return  $n$ 
5   end
6   else if  $i < \text{size}(n.\text{left}) + 1$  then
7      $n \leftarrow n.\text{left}$ 
8   end
9   else
10     $i \leftarrow i - \text{size}(n.\text{left}) + 1$ 
11     $n \leftarrow n.\text{right}$ 
12  end
13 end
14 return  $\text{NULL}$                                      // Posizione non valida

```

La size influenza i costi?

L'introduzione dell'attributo *size* in ogni nodo non altera la complessità temporale delle operazioni, che rimangono nell'ordine di $O(\log n)$.

Rotazioni Le rotazioni (semplici o doppie) sono operazioni locali che coinvolgono un numero costante di nodi. Poiché la *size* di un nodo è

$$\text{size}(n) = \text{size}(n.\text{left}) + \text{size}(n.\text{right}) + 1$$

il ricalcolo del valore richiede tempo costante $O(1)$. Nelle rotazioni semplici 3.1, gli unici nodi soggetti ad aggiornamento sono u e v ; nelle rotazioni doppie 3.2, l'aggiornamento interessa i nodi v , z e w . Di conseguenza, il costo aggiuntivo per l'aggiornamento delle *size* durante il ribilanciamento è $O(1)$.

Operazioni di Insert e Delete Durante le fasi di inserimento o rimozione, il cammino percorso dalla radice verso il nodo target ha lunghezza proporzionale all'altezza dell'albero che sappiamo essere $O(\log n)$.

- **Insert:** Ogni nodo lungo il cammino dal nodo inserito fino alla radice vede la propria *size* incrementata di un'unità.
- **Delete:** Ogni nodo lungo il cammino del padre del nodo rimosso fino alla radice vede la propria *size* decrementata di un'unità.

Tali aggiornamenti dunque non alterano il costo delle operazioni che rimane $O(\log n)$.

Operazioni di Join e Split Nell'operazione di Join, il valore *size* del nuovo nodo di giunzione i viene calcolato come $\text{size}(i) = \text{size}(x) + \text{size}(S_{\min}) + 1$, dove $S_{\min}$ è l'albero con rank minore tra S_1 e S_2 . Successivamente, risalendo il cammino da i verso la radice dell'albero, si incrementano le *size* dei nodi sul cammino di una quantità pari a $(\text{size}(S_{\min}) + 1)$. Poiché la risalita è limitata dall'altezza dell'albero, il costo è $O(\log n)$.

Siccome l'operazione di `Split` è basata sull'operazioni di `Join`, e il costo di quest' ultime rimane invariato anche la complessità dello `Split` resta invariata.

4.2 Rappresentazione del Tour

Nella formulazione classica degli Euler Tour Tree (ETT), un albero viene linearizzata tramite una visita in profondità (Depth-First Search), che produce una sequenza di archi orientati. In un contesto di foresta dinamica, dove le operazioni di `link` e `cut` modificano continuamente la struttura del grafo. Poiché gli archi (u, v) sono per loro natura entità effimere soggette a creazione e distruzione, il loro utilizzo come punti di accesso ai vertici risulta intrinsecamente fragile. La rimozione di un arco comporterebbe la potenziale invalidazione dei puntatori memorizzati nelle strutture di supporto che introdurremo di seguito, richiedendo onerose operazioni di ricerca per ripristinare la coerenza dei dati.

Per garantire la persistenza dei riferimenti, si introduce nella sequenza euleriana un elemento speciale per ogni vertice $v \in V$, denominato nodo identità, denotato formalmente come (v, v) .

L'integrazione di questi nodi risponde a due esigenze fondamentali:

- il nodo (v, v) funge da ancora fissa all'interno del tour. Una volta istanziato, esso non viene mai rimosso dalla struttura, indipendentemente dalle variazioni topologiche del grafo.
- un vertice isolato v costituisce un tour euleriano composto dal solo nodo (v, v)

Strutture di supporto: le Tabelle Hash

Per garantire che le operazioni di `link`, `cut` e `rerooting` mantengano una complessità temporale logaritmica $O(\log n)$, è fondamentale eliminare la dipendenza dalla scansione lineare del ciclo euleriano. Per risolvere tale problema introduciamo la struttura dati *Hash Map* agisce come un dizionario che mappa chiavi univoche a riferimenti di memoria, permettendo operazioni di ricerca in tempo costante $O(1)$. Il principio di funzionamento si basa su una funzione di hash $h : U \rightarrow \{0, \dots, m-1\}$ che mappa un universo di chiavi U in un array di dimensione m . Per prevenire il degradamento delle prestazioni dovuto alle collisioni (quando $h(u) = h(v)$ per $u \neq v$), si adotta una famiglia di hash universale $\mathcal{H}$, che limita la probabilità di collisione tra due chiavi distinte a $1/m$. Nella struttura degli Euler Tour Trees, l'integrazione delle tabelle hash permette di passare rapidamente dall'albero originale alla posizione fisica nel RB-Trees tramite due mappature distinte:

- **mapNode:** Associa ogni nodo originale $v \in V$ all'indirizzo di memoria del un nodo identità. Questa mappa è essenziale per l'operazione di `rerooting`, consentendo di individuare rapidamente l'inizio del tour.

$$\text{mapNode}[F] \rightarrow \&(F, F)$$

- **mapEdge:** Associa una coppia ordinata di nodi (u, v) , che rappresenta un arco diretto univoco del tour, al corrispondente nodo nel RB-Trees. Questa mappatura trasforma la ricerca di un arco da

una scansione lineare $O(n)$ a un accesso diretto $O(1)$, rendendo efficienti le operazioni di *split* e *join* necessarie per il **cut** e il **link**.

4.3 Operazioni

isConnected

Per eseguire l'operazione **isConnected**(u, v) bisogna verificare che u e v siano nello stesso RB-Trees per farlo basta risalire l'albero prima da u e poi da v per verificare che la radice sia la medesima

Algorithm 12: isConnected

Input: Nodo u , nodo v

Output: True o False

```

1  $r \leftarrow \text{NULL}$ 
2  $p \leftarrow u$                                      // Risale da u
3
4 while  $p.\text{parent} \neq \text{NULL}$  do
5    $p \leftarrow p.\text{parent}$ 
6 end
7  $r \leftarrow p$ 
8  $p \leftarrow v$                                      // Risale da v
9
10 while  $p.\text{parent} \neq \text{NULL}$  do
11    $p \leftarrow p.\text{parent}$ 
12 end
13 return  $r == p$ 

```

Rerooting

Per ottimizzare l'operazione ausiliaria di **rerooting**(T, v), rendendola computazionalmente efficiente, si sfruttano le proprietà degli alberi binari di ricerca bilanciati (Red-Black Trees) in combinazione con una tabella hash per l'accesso diretto ai nodi. Tale approccio permette di ridurre la complessità dell'operazione da $O(n)$ a $O(\log n)$.

La procedura si articola nelle seguenti fasi:

1. **Accesso alla sequenza:** Si interroga la hash map $\text{mapNode}[v]$ per ottenere il puntatore al nodo dell'RB-Tree che memorizza un arco uscente da v , denotato come $e_{\text{start}} = (v, c)$. Questa operazione garantisce la localizzazione del punto di taglio in tempo costante $O(1)$.
2. **Decomposizione (Split):** Si esegue l'operazione di $\text{split}(T, e_{\text{start}})$ sulla sequenza originale. Tale operazione scinde l'albero T in tre componenti distinte:
 - ▶ T_L : sottosequenza contenente tutti gli archi che precedono e_{start} nel tour corrente;
 - ▶ e_{start} : il nodo stesso identificato come punto di rotazione;
 - ▶ T_R : sottosequenza contenente gli archi successivi a e_{start} .
3. **Rimozione della vecchia apertura:** Si estrae l'elemento situato all'estrema destra di T_L . Tale elemento, che denotiamo con P , rappresenta l'arco di inizio del tour originale.

4. **Ricomposizione della sequenza (Join):** Si procede alla creazione della nuova sequenza T_{new} invertendo l'ordine relativo dei blocchi estratti. Utilizzando P come elemento di giunzione, si esegue:

$$T_{new} = \text{join}(T_R, P, T_L)$$

5. **Finalizzazione del ciclo:** Per garantire la coerenza della struttura euleriana e ristabilire il nodo v come radice si inserisce e_{start} come elemento minimo (all'inizio) della sequenza T_{new} .

Link

L'operazione di **link**(u, v) permette di unire due alberi disgiunti attraverso l'introduzione di un nuovo arco non orientato, rappresentato dalla coppia di archi diretti (u, v) e (v, u). Per garantire un'efficienza logaritmica $O(\log n)$, l'operazione sfrutta le proprietà dell'Euler Tour Tree e l'operazione ausiliaria di *rerooting*. La procedura si articola nelle seguenti fasi:

1. **Controllo di connettività:** Come prima misura di sicurezza, si interroga la funzione $\text{isConnected}(u, v)$. Se i due nodi appartengono già alla stessa componente connessa, l'operazione termina immediatamente per evitare la creazione di cicli.
2. **Normalizzazione delle radici (Reroot):** Si eseguono le operazioni di $\text{reroot}(u)$ e $\text{reroot}(v)$ sulle rispettive componenti. Questa fase è fondamentale: modifica la rappresentazione dell'Euler Tour in modo che il tour del primo albero inizi nel nodo u e quello del secondo inizi nel nodo v , predisponendo le sequenze per una giunzione lineare coerente.
3. **Identificazione delle radici dei sottoalberi:** Si interroga la hash map dei nodi (mapNode) per ottenere i puntatori ai nodi dell'RB-Tree corrispondenti a u e v . Tramite la funzione findRoot , si risale alle radici correnti delle due sequenze, che denotiamo come rootU e rootV .
4. **Allocazione e registrazione dei nuovi archi:** Si istanziano dinamicamente due nuovi nodi nell'RB-Tree per rappresentare l'arco nei due sensi di percorrenza: $uv = (u, v)$ e $vu = (v, u)$. I puntatori a tali nodi vengono contestualmente inseriti nella hash map mapEdge per consentire futuri accessi in tempo costante $O(1)$.
5. **Composizione della sequenza (Join):** Si procede alla concatenazione delle due sequenze principali utilizzando l'arco diretto uv come elemento di giunzione (perno). Viene eseguita l'operazione:

$$\text{tempRoot} = \text{join}(\text{rootU}, uv, \text{rootV})$$

In questo modo, l'arco uv collega la fine del tour di u (che ora termina in u grazie al reroot) con l'inizio del tour di v .

6. **Chiusura del ciclo euleriano:** Per ristabilire la struttura ciclica dell'Euler Tour complessivo e permettere il ritorno dal secondo sottoalbero verso il primo, si inserisce l'arco di ritorno vu . Questo nodo viene inserito come elemento massimo (ovvero in ultima posizione a destra) all'interno della sequenza appena unita tramite l'operazione:

$$\text{insertMax}(\text{tempRoot}, vu)$$

Al termine di questa sequenza di operazioni, i due alberi originari risulteranno fusi in un unico RB-Tree, il cui tour euleriano rappresenta l'albero unito con radice complessiva nel nodo u .

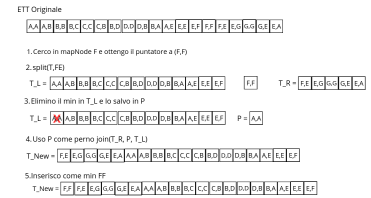


Figura 4.1: Procedura di rerooting

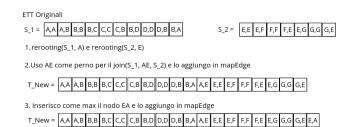


Figura 4.2: Procedura di link

cut

L'operazione di $\text{cut}(u, v)$ permette di separare un albero in due alberi distinti attraverso la cancellazione dell'arco (u, v) e del suo rispettivo arco di ritorno (v, u) . Per garantire un'efficienza logaritmica $O(\log n)$, l'operazione viene implementata sfruttando le strutture dati precedentemente introdotte (Euler Tour Tree e Red-Black Tree come motore di sequenza) e si articola nelle seguenti fasi:

1. **Verifica ed estrazione dei puntatori:** Si interroga la hash map mapEdge per verificare l'esistenza dell'arco (u, v) . Se presente, si ottengono in tempo costante $O(1)$ i puntatori ai nodi dell'RB-Tree corrispondenti: $uv = \text{mapEdge}[u, v]$ e $vu = \text{mapEdge}[v, u]$.
2. **Prima decomposizione (Split):** Si individua la radice dell'RB-Tree contenente uv e si esegue l'operazione di $\text{split}(T, uv)$. Questa operazione scinde la sequenza originale in due sottoalberi:
 - T_L : sottosequenza contenente gli elementi che precedono uv ;
 - T_R : sottosequenza contenente gli elementi che succedono uv .

Il nodo uv viene rimosso dal tour e isolato.

3. **Risoluzione dell'orientamento dell'arco:** L'ordine relativo tra uv e vu nell'Euler Tour non è noto a priori. Si determina quindi la posizione di vu controllando se la sua radice coincide con la radice di T_R :

a) **Scenario A (vu si trova in T_R):**

- **Split secondario:** Si esegue $\text{split}(T_R, vu)$, separando T_R in T_{middle} (elementi tra uv e vu) e T_{right} (elementi successivi a vu). Il nodo vu viene isolato.
- **Estrazione del perno:** Si individua il nodo minimo di T_{right} , denominato p_{old} , che rappresenta l'arco di rientro alla radice.
- **Riciclo del nodo:** Si cancella p_{old} dalla mappa, lo si scollega fisicamente tramite $\text{detach}(T_L, p_{old})$, e lo si reinserisce in mappa aggiornato.
- **Ricomposizione (Join):** Si uniscono le sequenze rimanste inserendo il perno riciclato: $\text{join}(T_L, p_{old}, T_{right})$. Il sottoalbero isolato che rappresenta il ramo tagliato sarà T_{middle} .

b) **Scenario B (vu si trova in T_L):**

- **Split secondario:** Si esegue $\text{split}(T_L, vu)$, separando T_L in T_{left} (elementi precedenti a vu) e T_{middle} (elementi tra vu e uv). Il nodo vu viene isolato.
- **Estrazione del perno:** Si individua il nodo minimo di T_{left} , denominato p_{old} .
- **Riciclo del nodo:** Analogamente allo scenario precedente, si rimuove p_{old} dalla mappa, si effettua il $\text{detach}(T_L, p_{old})$ e lo si reinserisce aggiornato.
- **Ricomposizione (Join):** Si uniscono le sequenze speculari inserendo il perno: $\text{join}(T_R, p_{old}, T_{left})$. Il sottoalbero isolato che rappresenta il ramo tagliato sarà in questo caso T_{middle} .

4. **Rimozione definitiva e deallocazione:** Infine, si eliminano definitivamente le chiavi $\{u, v\}$ e $\{v, u\}$ dalla mappa mapEdge e si libera la memoria deallocando fisicamente i puntatori dei nodi uv e vu .

Al termine della procedura, la sequenza T_{middle} conterrà l'Euler Tour del sottoalbero distaccato (radicato in v o u a seconda dell'orientamento

originario), mentre la sequenza ricostruita tramite la join rappresenterà il resto dell'albero originale privato di tale componente.

4.4 Codice

La formalizzazione matematica e l'analisi algoritmica presentate nei paragrafi precedenti definiscono la struttura in ogni minimo dettaglio implementativo. La scomposizione delle operazioni d'alto livello (*link*, *cut*, *rerooting*) nelle primitive cromatiche e strutturali dei Red-Black Tree è stata condotta a un livello di granularità tale da rendere la successiva fase di sviluppo software una conversione diretta, dal modello teorico al paradigma di programmazione.

Per questo motivo, si rimanda l'analisi del codice sorgente completo, scritto interamente in linguaggio C++ e organizzato in classi modulari e riutilizzabili, direttamente alla consultazione del repository ufficiale del progetto. L'implementazione pratica degli Euler Tour Trees (ETT), comprensiva dell'architettura dei RB-Tree con annesse operazioni di *split* e *join* e dei relativi test di validazione, è liberamente accessibile al seguente indirizzo:

<https://github.com/VittorioPorri/Tesi>

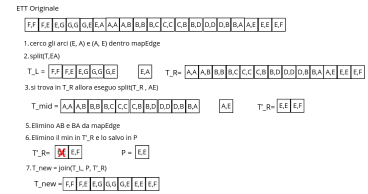


Figura 4.3: Procedura di cut

Desidero esprimere la mia più profonda gratitudine al Professor Luciano Gualà, per la guida autorevole, la disponibilità e il costante sostegno dimostrati durante la stesura di questo elaborato.

Un ringraziamento speciale va alla mia stupenda fidanzata. Senza di te, probabilmente, oggi festeggerei una laurea in chimica o, forse, non avrei mai tagliato questo incredibile traguardo. Grazie per avermi sempre sostenuto, supportato, ascoltato e capito in ogni singolo momento, soprattutto in quelli più difficili. Grazie per essere la mia ancora, la mia complice, la mia migliore amica e la mia amante. Sei un pilastro fondamentale della mia vita; lo sei stata lungo tutto questo percorso, lo sei oggi e spero con tutto il cuore che lo sarai anche nel nostro futuro. Sei speciale e ti amo tanto.

A mio papà, che si è sempre fatto in quattro per permettermi di studiare senza preoccupazioni e concentrarmi sul portare a termine questo obiettivo. Grazie per averci provato con tutto te stesso a insegnarmi tutte le cose stupende che sai fare... anche se, diciamocelo papà, non sei proprio l'insegnante più paziente del mondo! Sappiamo entrambi che quando facciamo qualcosa insieme io sono un po' imbranato e tu pretendi la perfezione, quindi finisci sempre per fare tutto da solo. E quando provo a farti delle domande perché non ci capisco nulla, puntualmente non mi rispondi; so che a volte sono domande sceme, ma a volte le cose semplici sono le più fondamentali. Grazie di cuore per tutto quello che fai per me.

Alla mamma, che con la scusa di un piccolo favore ha saputo regalarmi quelle pause necessarie per svagare la mente durante le sessioni di studio. E anche se non lo ammetterà mai davanti a me, facendo sempre la parte di quella "dura", so benissimo che quando non ci sono è super fiera di quello che faccio e non perde occasione per vantarsi di me con le sue amiche. Grazie per il tuo amore silenzioso.

Al nonno, che è un vero guerriero per essere qui oggi. Ne hai passate tante negli ultimi mesi, ma nonostante tutto hai sempre vinto. Grazie per avermi permesso di assisterti e supportarti, e grazie per avermi insegnato un mondo di cose sull'orto. Ti prometto che, proprio come ho fatto quest'anno, anche nei primi anni verrò io a fare i lavori più pesanti... perché ormai, lo sappiamo, hai una certa età!

Alla nonna, che mi ha sempre preparato piatti deliziosi per saziare il mio stomaco perennemente affamato, per avermi sempre coccolato e trattato come un principe.

Alla zia, per avermi accompagnato nella difficilissima scelta dell'abito da indossare in questa occasione così importante. Grazie per la pazienza infinita, per i tuoi preziosi consigli di stile e per aver saputo gestire la mia indecisione con il sorriso.

Ai miei compagni di corso: con voi ho condiviso momenti indimenticabili. Ci siamo aiutati e sostenuti a vicenda nei momenti più duri; abbiamo riso, sofferto e riso di nuovo, sempre fianco a fianco. Finalmente, siamo arrivati quasi tutti insieme a questo traguardo. Un pensiero speciale va anche a quelle quattro mura del laboratorio di informatica, che per anni ci hanno ospitato e fatto sentire a casa.

Infine, grazie agli amici della palestra. Condividere con voi fatica e sudore mi ha motivato a dare di più, sia dentro che fuori dalla sala pesi. Devo fare una confessione: continuo a pensare che la palestra sia un po' noiosa, ma se ad attendermi ci sono persone speciali e simpatiche come voi ci vengo sempre volentieri... e magari faccio pure finta di allenarmi!

Grazie a tutti voi.

Bibliography

Here are the references in citation order.

- [1] Monika Rauch Henzinger e Valerie King. «Randomized Dynamic Graph Algorithms with Polylogarithmic Time per Operation». In: *Journal of the ACM (JACM)* 46.4 (1999), pp. 502–516 (citato a pagg. 1, 3).
- [2] Robert Endre Tarjan. *Data Structures and Network Algorithms*. Vol. 44. CBMS-NSF Regional Conference Series in Applied Mathematics. Philadelphia, PA: Society for Industrial e Applied Mathematics (SIAM), 1983 (citato a pagg. 1, 11).
- [3] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. Chapter 14: Augmenting Data Structures. MIT Press, 2009 (citato a pag. 17).

